

Q1. Requirements Analysis

Functional Requirements

1. Lock Acquisition

The system must allow a client to acquire a lock on a shared resource. Before granting a lock, the system checks whether the resource is already locked. If the resource is available, a unique token is generated and assigned to the client. This prevents multiple clients from accessing the same resource simultaneously.

2. Lock Release

The system must provide a mechanism to release locks after the client completes its operation. This ensures that resources become available for other clients and prevents unnecessary resource blocking.

3. Lock Renewal

Long-running operations may require more time than the original lock duration. Therefore, the system must support lock renewal by extending the lock expiration time while maintaining ownership validation.

4. Lock Expiration

Every lock should have a Time-To-Live (TTL). If a client fails or becomes unreachable, the lock should automatically expire to prevent deadlocks and resource starvation.

5. Ownership Verification

Only the client that owns a lock should be allowed to renew or release it. Ownership is verified using unique lock tokens.

6. Monitoring and Inspection

Administrators should be able to view active locks and their metadata for debugging and monitoring purposes.

7. Distributed Coordination

The locking service should coordinate resource access across multiple distributed applications and services.

Non-Functional Requirements

Scalability

The system should handle thousands or millions of lock requests efficiently.

Availability

The service should remain operational even when some nodes fail.

Reliability

Lock information should remain accurate and consistent.

Consistency

Only one valid lock should exist for a resource at any given time.

Fault Tolerance

The system should recover from failures without losing critical lock information.

Security

Unauthorized clients must not be able to manipulate locks.

Performance

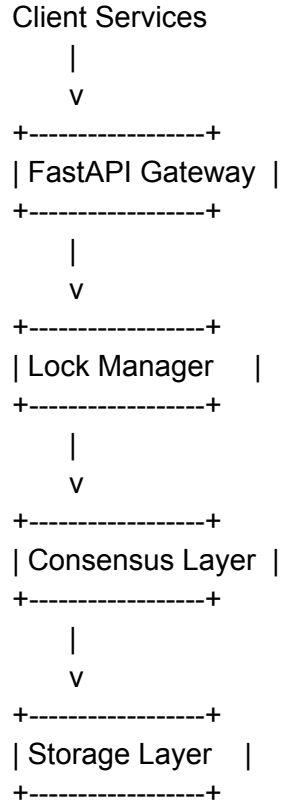
Lock operations should execute with minimal latency.

Importance for LockSync

These requirements ensure that LockSync provides reliable resource coordination while preventing race conditions, deadlocks, and inconsistent data states in distributed systems.

Q2. System Architecture Design

High-Level Architecture



Components

Client Services

Applications that request locks before accessing shared resources.

FastAPI Gateway

Acts as the entry point for all lock-related requests. It validates requests and routes them to the Lock Manager.

Lock Manager

Core component responsible for:

- Lock acquisition
- Lock renewal
- Lock release
- Expiration handling
- Ownership verification

Consensus Layer

In a production environment, algorithms such as Raft would be used to maintain consistency across multiple nodes and perform leader election.

Storage Layer

Stores lock metadata, ownership details, expiration information, and replication logs.

Working Process

1. Client sends a lock request.
2. FastAPI receives the request.
3. Lock Manager checks resource availability.
4. Lock is granted if available.
5. Metadata is stored.
6. Client performs operation.
7. Lock is renewed or released.
8. Expired locks are automatically removed.

This architecture ensures scalability, consistency, and reliability.

Q3. Lock Acquisition and Release

Lock Acquisition

The lock acquisition process begins when a client requests access to a resource.

Steps

1. Client sends resource ID and owner ID.
2. System checks whether the resource is already locked.
3. If not locked, a UUID token is generated.
4. Expiration time is calculated using TTL.
5. Lock metadata is stored.
6. Token is returned to the client.

Example

Resource: inventory_101

Owner: service_A
Token: abc123xyz
TTL: 30 seconds

Lock Renewal

If the operation takes longer than expected:

1. Client sends renewal request.
2. Token validation occurs.
3. Expiration time is extended.
4. Updated metadata is stored.

Lock Release

After completion:

1. Client sends release request.
2. System validates ownership.
3. Lock entry is removed.
4. Resource becomes available.

Lock Expiration

Each lock has a TTL value. When TTL expires:

1. Lock becomes invalid.
2. Cleanup mechanism removes expired lock.
3. Resource becomes available.

Deadlock Prevention

Deadlocks are prevented through:

- Automatic lock expiration
- Ownership validation
- Lock timeout mechanism
- Resource cleanup process

These mechanisms ensure efficient resource utilization.

Q4. Database Design

Database Selection

MongoDB is selected because it offers:

- High scalability
- Flexible schema
- Fast read/write operations
- Built-in replication support

Collection Structure

Locks Collection

Field	Type	Description
_id	String	Unique document ID
resource_id	String	Resource identifier
owner_id	String	Lock owner
token	String	Ownership token
created_at	DateTime	Creation timestamp
expires_at	DateTime	Expiration timestamp
status	String	Active/Released

Sample Document

```
{
  "_id": "1",
  "resource_id": "resource_101",
  "owner_id": "service_A",
  "token": "abc123xyz",
  "created_at": "2026-06-15",
  "expires_at": "2026-06-15T12:30:00",
  "status": "active"
}
```

Indexing Strategy

Indexes should be created on:

- resource_id
- token
- expires_at

TTL Index

MongoDB TTL indexes can automatically delete expired locks, reducing manual cleanup operations.

Advantages

- Fast lookup
 - Efficient expiration handling
 - Easy horizontal scaling
 - Replication support
-

Q5. Algorithm and Implementation

Locking Algorithm

LockSync uses a token-based locking mechanism.

Algorithm

Acquire Lock

1. Receive lock request.
2. Check resource availability.
3. Verify existing lock expiration.
4. Generate UUID token.
5. Store lock metadata.
6. Return token.

Renewal Algorithm

1. Receive token.

2. Validate ownership.
3. Update expiration time.
4. Return success.

Release Algorithm

1. Receive token.
2. Verify ownership.
3. Remove lock.
4. Return success.

Python Implementation

The implementation uses:

- FastAPI
- UUID token generation
- Threading lock
- Time-based expiration
- Dictionary-based storage

APIs Implemented

POST /acquire-lock

POST /renew-lock

POST /release-lock

GET /locks

Benefits

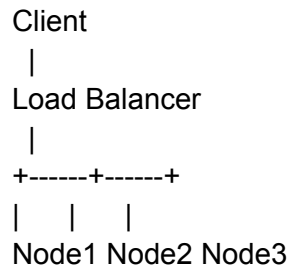
- Simple implementation
- Low latency
- Easy to understand
- Extensible architecture

Q6. Scalability and Fault Tolerance

Scalability

Horizontal Scaling

Multiple LockSync instances can be deployed behind a load balancer.



Sharding

Resources can be partitioned across nodes.

Example:

Node 1 → Resources A-H

Node 2 → Resources I-P

Node 3 → Resources Q-Z

This distributes workload efficiently.

Load Balancing

Tools such as Nginx and HAProxy can distribute incoming requests across multiple nodes.

Fault Tolerance

Replication

Leader node replicates lock data to replica nodes.

Leader Election

If the leader fails, the Raft algorithm elects a new leader automatically.

Crash Recovery

Expired locks are automatically released.

Network Partition Handling

Majority voting ensures consistency during network failures.

Data Consistency

Only one valid lock is maintained for each resource, preventing conflicting updates.

Conclusion

By combining horizontal scaling, replication, leader election, expiration handling, and consensus algorithms, LockSync can evolve into a highly available, fault-tolerant distributed locking system capable of supporting large-scale enterprise applications while maintaining consistency and reliability.